Unit – 1
Introduction and Types of NoSQL Databases

1.1 Introduction to NoSQL Databases

- NoSQL databases, or "Not Only SQL," represent a diverse category of database management systems designed to address the limitations of traditional relational databases.
- NoSQL databases are defined as non-tabular databases that handle data storage differently as compared to relational tables. Unlike SQL databases, NoSQL databases are schema- less and provide a more flexible data model, allowing for the storage and retrieval of large volumes of unstructured or semi-structured data. They are particularly well-suited for handling dynamic and rapidly evolving data in distributed and scalable environments.
- NoSQL databases are gaining traction for real-time cloud, web, and big data applications due to their key features:

1. Multiple Data Model Compatibility:

- NoSQL supports various data models, allowing flexibility in handling unstructured, semi-structured, and structured data.
- Ideal for Agile development, as it accommodates different data models without the need for separate databases.

2. Enhanced Scalability and Availability:

- NoSQL offers simplified scalability with a serverless, peer-to-peer architecture, allowing for horizontal scaling and improved performance.
- Sharding enables efficient handling of massive data volumes, and auto-replication ensures high availability in case of failures.

3. Global Data Distribution:

- Advanced NoSQL databases facilitate global data distribution by operating across multiple cloud regions and data centers.
- Contrastingly, relational databases often rely on centralized applications, while NoSQL minimizes wait times by distributing data globally.

4. Minimal Downtime:

- NoSQL databases feature robustness and minimal downtime
- Serverless architecture and data replication across nodes ensure business continuity, with alternative nodes providing access in case of a malfunction.
- Can NoSQL completely replace relational databases?

- No, NoSQL cannot entirely replace relational databases. The choice between the two depends on the specific needs of an organization. NoSQL is preferred for handling diverse data types in cloud and web applications with a broad and rapidly growing user base. Its flexibility, multi-modality, scalability, availability, and global distribution make it ideal for database.

1.2 Types of NoSQL Databases

There are four main types of NoSQL databases, each catering to specific use cases:

1. Document-oriented Databases:

   - Store data in flexible, JSON-like documents.

   - Examples: MongoDB, CouchDB.

2. Key-Value Stores:

   - Simplest NoSQL model, storing data as key-value pairs.

   - Examples: Redis, DynamoDB.

3. Column-family Stores:

   - Organize data into columns rather than rows.

   - Examples: Apache Cassandra, HBase.

4. Graph Databases:

   - Designed for managing and querying interconnected data.

   - Examples: Neo4j, Amazon Neptune.

1.3 CAP Theorem

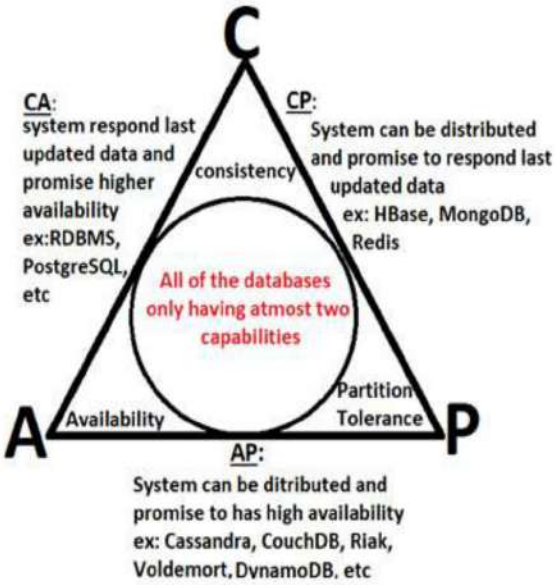
- The CAP theorem, originally known as the CAP principle, illuminates the inherent trade-offs in designing distributed systems with replication. It highlights the challenge of simultaneously achieving consistency, availability, and partition tolerance in such systems.

VPMP POLYTECHNIC

1. Consistency (C):
- All nodes in the system see thesame data at the same time.

2. Availability (A):
- Every request receives a response, without guarantee that it contains the most recent version of the information.

3. Partition Tolerance (P):
- The system continues to operate despite network partitions that may cause communication failures betweennodes.



- The CAP theorem asserts that distributed databases can prioritize at most two of these threeproperties. Database systems fall into categories based on their priorities:
- CA (Consistency and Availability): Prioritizing availability over consistency, responding withpossibly stale data. Examples include Cassandra, CouchDB, Riak, and Voldemort.
- AP (Availability and Partition Tolerance): Emphasizing availability over consistency, possiblyresponding with stale data. Distributed across nodes, designed to operate reliably despite network partitions. Examples include Amazon DynamoDB and Google Cloud Spanner.
- CP (Consistency and Partition Tolerance): Focusing on consistency over availability, responding with the latest updated data. Distributed across nodes, designed to operate reliably in the face of network partitions. Examples include Apache HBase, MongoDB, and Redis.
- It's crucial to note that database systems may exhibit different behaviors based on configurations and settings, influencing their consistency, availability, and partition tolerance. Even in the case of specialized databases like Neo4j, which prioritizes consistency and partition tolerance over availability, the CAP theorem remains relevant. In situations of network partition or failure, Neo4j sacrifices availability to maintain consistency.
- System architects must make trade-offs based on the application's requirements and the characteristics of the underlying NoSQL database.

VPMP POLYTEC

1.4 Consistency in NoSQL

- In the context of NoSQL databases, consistency refers to the level of agreement between multiple copies of data within a distributed system. Unlike traditional relational databases, which typically prioritize strong consistency (ensuring that all copies of data are always synchronized), NoSQL databases often offer various levels of consistency to achieve better performance, scalability, and fault tolerance.

- Here are some common consistency models in NoSQL databases:

1. **Strong Consistency**: In a strongly consistent system, all replicas of data are always kept in sync. Any read operation will return the most recent write. Achieving strong consistency usually involves high latency and may limit scalability, as all nodes must agree on the state of data before proceeding.
2. **Eventual Consistency**: Eventual consistency allows for temporary inconsistencies between replicas but guarantees that given enough time and no further updates, all replicas will converge to the same state. This model prioritizes availability and partition tolerance over immediate consistency. It's often used in distributed systems where low latency and high availability are crucial.
3. **Causal Consistency**: Causal consistency ensures that if one operation causally precedes another, then all replicas will observe them in the same order. This model relaxes consistency constraints to improve performance while still maintaining a causal relationship between operations.
4. **Read Your Writes Consistency**: Also known as session consistency, this model guarantees that any data written by a client will be immediately readable by subsequent reads from the same client. It's a weaker form of consistency that suits many real-world applications where immediate consistency is not strictly required.
5. **Consistent Prefix Reads**: This model guarantees that clients will observe a prefix of all writes that have been acknowledged. It's weaker than strong consistency but stronger than eventual consistency. It ensures that clients can observe a consistent view of data even in the presence of concurrent updates.
6. **Monotonic Reads and Writes**: Monotonic consistency ensures that once a client observes a particular value for a data item, it will never see an older value for that item in subsequent reads. This guarantees progress in the system and helps prevent rollback of observed states.

VPMP POLYTECHNIC

1.5 Availability and Partition Tolerance

- In the context of NoSQL databases, availability and partition tolerance are two of the three properties in the CAP theorem, which states that in a distributed computer system, it is impossible to simultaneously achieve all three of the following:

1. Consistency: All nodes in the system have the same data at the same time.
2. Availability: Every request to the system receives a response, even if some nodes fail.
3. Partition Tolerance: The system continues to operate despite network partitions (communication failures) between nodes.

- Since partition tolerance is considered a must-have in distributed systems, NoSQL databases typically focus on the trade-off between consistency and availability. Here's how availability and partition tolerance are typically addressed in NoSQL databases:

1. High Availability: NoSQL databases are designed to ensure that data remains available even in the face of node failures or network partitions. They achieve this through replication, where multiple copies of data are stored on different nodes. In the event of a node failure, other nodes can still serve requests, maintaining system availability.
2. Replication: Most NoSQL databases support data replication across multiple nodes. Replicas can be synchronized asynchronously, allowing for eventual consistency while ensuring high availability. Some databases offer configurable consistency levels, allowing developers to choose the level of consistency they require based on their application's needs.
3. Partition Tolerance: NoSQL databases are designed to handle network partitions gracefully. When a network partition occurs, the database system continues to operate independently on each side of the partition. Once the network partition is resolved, mechanisms such as conflict resolution or reconciliation are employed to synchronize the data across the partitions.
4. Quorum-based Consistency: Many NoSQL databases use quorum-based techniques to ensure consistency and availability. For example, in a distributed system with data replicated across multiple nodes, a quorum of nodes must agree on read and write operations to ensure consistency. This allows the system to continue operating even if some nodes are unavailable due to network partitions.
5. Tunable Consistency: Some NoSQL databases offer tunable consistency levels, allowing developers to balance consistency and availability based on their application requirements. For example, developers can choose between strong consistency for critical data or eventual consistency for less critical data.

1.6 Comparisons between MongoDB, Cassandra, Neo4j and Redis

Criteria	MongoDB	Cassandra	Neo4j	Redis
Data Model	Document-oriented (BSON)	Column-family	Graph	Key-value store
Query Language	MongoDB Query Language (MQL)	CQL (Cassandra Query Language)	Cypher Query Language	N/A (Command-based)
Schema	Dynamic schema	Dynamic schema	Schema-based (flexible)	Schemaless
Scalability	Horizontal scaling through sharding	Horizontal scaling	Horizontal scaling	Horizontal scaling
Consistency	Tunable consistency (Eventual to Strong)	Tunable consistency (Eventual to Strong)	Strong consistency	Eventual consistency

Partition Tolerance	Yes	Yes	Yes	Yes
Use Case Focus	General-purpose, document storage	Wide-column store, time-series data	Graph-based, relationships	Cache, real-time analytics
Indexing	Multiple types, including compound and geo-spatial indexes	Secondary indexes	Indexing based on relationships	Limited indexing options
Query Performance	Good	Excellent	Excellent for graph queries	Very fast (in-memory storage)
ACID Transactions	Supports ACID transactions	Limited support for transactions	Supports ACID transactions	Supports transactions through multi commands
Community Support	Large and active community	Active community	Active community	Active community
Use Cases	Content management systems, e-commerce, real-time analytics	Time-series data, logging, IoT	Social networks, frauddetection	Caching, message broker

VPMP POLYTECHNIC

1.7 Use cases for different NoSQL databases

- Different NoSQL databases are designed with specific use cases and requirements in mind. Here are some common NoSQL databases and their typical use cases:

1. MongoDB:

- Document-Oriented: MongoDB stores data in flexible, JSON-like documents, making it suitable for applications with evolving schemas or semi-structured data.
- Real-time Analytics: MongoDB's flexible schema and support for ad-hoc queries make it well-suited for real-time analytics applications where data needs to be ingested and analyzed quickly.

2. Cassandra:

- Distributed Data: Cassandra is designed for scalability and high availability, making it ideal for distributed data scenarios where data needs to be replicated across multiple nodes.
- Time-Series Data: Cassandra's ability to handle large volumes of time-series data with high write throughput makes it suitable for use cases such as IoT telemetry, sensor data, and event logging.

3. Redis:

- Caching: Redis's in-memory data store and support for various data structures (e.g., strings, hashes, lists) make it ideal for caching frequently accessed data and speeding up application performance.
- Session Management: Redis's fast read and write operations make it well-suited for storing session data in web applications, enabling quick retrieval and updates.

4. Elasticsearch:

- Full-text Search: Elasticsearch is optimized for full-text search, making it ideal for applications requiring advanced search capabilities, such as e-commerce platforms, content management systems, and log analysis.
- Log and Event Data: Elasticsearch's ability to index and analyze large volumes of log and event data in real-time makes it suitable for monitoring, analytics, and log aggregation use cases.

5. Neo4j:

- Graph Data: Neo4j is a graph database designed for storing and querying graph data, making it suitable for applications with complex relationships, such as social networks, recommendation engines, and fraud detection systems.
- Network Analysis: Neo4j's graph-based query language (Cypher) and optimized graph algorithms make it well-suited for network analysis and graph-based computations.

6. Amazon DynamoDB:

- Scalable Web Applications: DynamoDB is a fully managed NoSQL database service offered by AWS, making it suitable for building scalable web applications that require low-latency, high throughput, and automatic scaling.
- Ad Tech: DynamoDB's ability to handle high volumes of read and write

VPMP POLYTECHNIC

operations with consistent performance makes it suitable for ad tech platforms, where real-time bidding and personalized ad targeting are critical.

1.8 Factors influencing choice of database

- **Data Model:** Choose based on whether document-oriented, key-value, column-family, or graph data models are most suitable for the application.
- **Scalability Requirements:** Consider the need for horizontal scalability and how well the database can distribute data across multiple nodes.
- **Consistency and Availability Needs**: Assess the trade-offs between consistency andavailability based on the application's requirements.
- **Use Case Characteristics**: Tailor the choice of database to the specific demands of the application, such as read and write patterns, data complexity, and relationships.
- **Development and Operational Considerations:** Evaluate factors like ease of development, community support, and operational overhead when choosing a NoSQL database for a project.
- **Query Flexibility**: Consider the types of queries your application needs to support. Some NoSQL databases offer rich query languages and indexing capabilities, enabling complex ad-hoc queries and full-text search, while others may have limited querying capabilities.
- **Fault Tolerance and High Availability**: Assess your tolerance for node failures and network partitions. Look for features such as data replication, automatic failover, and consistency guarantees to ensure data availability and reliability.
- **Community and Ecosystem**: Consider the size and activity of the community surrounding the NoSQL database, as well as the availability of libraries, tools, and documentation. A vibrant community can provide valuable support and resources for development and troubleshooting.
- **Operational Overhead**: Evaluate the operational complexity and overhead associated with deploying and managing the NoSQL database. Consider factors such as deployment options (cloud-managed vs. self-hosted), data migration capabilities, backup and recovery mechanisms, and monitoring and management tools.
- **Cost**: Consider the total cost of ownership (TCO) associated with the NoSQL

VPMP POLYTECHNIC

database, including licensing fees, hardware/software costs, operational expenses, and scalability costs. Compare the pricing models of different databases, such as pay-as-you-go, subscription-based, or open-source.

- In conclusion, understanding the nuances of NoSQL databases, their types, trade-offs, and specific use cases is crucial for making informed decisions in selecting the right database solution for diverse applications.

VPMP POLYTECHNIC